Bitácora de prompts — Compilador de álgebra relacional a SQL

Resumen

Nº	Fecha	Fase	Tema
1	20/10/2026	Propuesta	Primera arquitectura del compilador
2	20/10/2026	Propuesta	Diapositivas para presentar la idea
3	20/10/2026	Conceptos y diseño	Qué es un AST; uso de símbolos griegos
4	20/10/2026	Planificación	Cómo se construye el compilador con Coco/R
5	20/10/2026	Diseño del lenguaje	Adoptar la sintaxis de RelaX
6	20/10/2026	Diseño del lenguaje	Alcance según el material de Base de Datos 1
7	20/10/2026	Etapa 1: scanner y parser	Gramática de la versión 1 y pruebas
8	01/11/2026	Entorno	Descarga de Coco.exe
9	01/11/2026	Entorno	Interpretación de la salida de Coco/R
10	01/11/2026	Entorno	Error al instalar .NET
11	01/11/2026	Entorno	Instalación de .NET 10
12	01/11/2026	Pruebas	Resultado de las pruebas de la etapa 1
13	01/11/2026	Etapa 2: AST	Construcción del árbol de sintaxis abstracta
14	01/11/2026	Documentación	Archivo de contexto para Claude Code
15	01/11/2026	Etapas 3 y 4	Análisis semántico y generador de SQL

Propuesta, diseño del lenguaje y etapa 1

Prompt 1 — Primera arquitectura del compilador

Fecha: 20/10/2026 · **Fase:** Propuesta

Prompt original

> Quiero que me ayudes a desarrollar una idea para un proyecto para la materia de compiladores. Nos dieron como trabajo integrador el desarrollar nuestro propio compilador con la función que se nos ocurra. A mi grupo se le ocurrio desarrollar un compilador de algebra relacional que compile a SQL (en principio). Nuestra idea surgió gracias a que una integrante del grupo es ayudante de la materia base de datos 1, y durante el cursado de esa materia los alumnos tienen problemas durante la unidad de algebra relacional porque la herramienta que recomienda la catedra requiere obligatoriamente tener una computadora para ejectuarla. Se nos ocurrió hacer un compilador con funciones parecidas a Relax pero cuyo compilador este desarrollado por nosotros mismos y con una interfaz responsive que permita el uso tanto en computadora como en telefono, de forma que sea mas accesible para los alumnos de base de datos en próximos años. En un principio tenemos que presentarle al profesor de compiladores una suerte de "presentación del proyecto" donde le expliquemos de forma resumida que es lo que vamos a hacer. Tenes que centrarte sobre todo en la parte del compilador y no tanto en la interfaz grafica o que vaya a ser utilizado para dar clases en otra cátedra. Por lo tanto, agradecería que nos des un primer pantallazo de como podríamos armar el compilador de forma que sea fácil de explicar, entender e implementar utilizando coco/r e integrandolo en un futuro a una app web responsive

Resultado: propuso el flujo scanner → parser → AST → análisis semántico → generador de SQL, una gramática inicial con niveles de precedencia y la idea de encapsular el compilador como una biblioteca independiente de la interfaz. Advirtió sobre el alcance de RelaX y sobre la necesidad de que la gramática sea LL(1).

Prompt 2 — Diapositivas para presentar la idea

Fecha: 20/10/2026 · **Fase:** Propuesta

Prompt original

> Armalo como diapositivas muy sencillas, no hace falta que profundices nada todavía. Con que se entienda el flujo del compilador y como se relacionan las distintas partes de un compilador entre ellas

Resultado: una presentación de 7 diapositivas: la idea, el flujo del compilador con la tabla de símbolos, scanner y parser, análisis semántico, generación de SQL e integración web.

Prompt 3 — Qué es un AST y uso de símbolos griegos

Fecha: 20/10/2026 · **Fase:** Conceptos y diseño

Prompt original

> que es AST? Además me gustaría que la presentación y el lenguaje en un futuro (al menos para el usuario, de forma grafica) muestre los simbolos griegos y no las palabras (en vez de escribir pi, poner el simbolo griego pi)

Resultado: explicó la diferencia entre la lista de tokens y el árbol, por qué se llama "abstracto" y cómo se construye con acciones semánticas en Coco/R. Actualizó las diapositivas y propuso que el scanner acepte la palabra y el símbolo como el mismo token. Advirtió que Coco/R necesita la entrada en UTF-8 para leer los símbolos.

Prompt 4 — Cómo se construye el compilador con Coco/R

Fecha: 20/10/2026 · **Fase:** Planificación

Prompt original

> Ahora quiero empezar con el desarrollo del sistema, empecemos por el compilador que es lo que se va a evaluar. Quiero que me ayudes a entender paso a paso como se constuye el compilador usando COCO/R

Resultado: una guía en 9 pasos: estructura del `.atg`, scanner, parser como reconocedor, condición LL(1), atributos y acciones semánticas, tabla de símbolos, análisis semántico, generación de SQL y símbolos griegos. Recomendó construir el compilador por capas.

Prompt 5 — Adoptar la sintaxis de RelaX

Fecha: 20/10/2026 · **Fase:** Diseño del lenguaje

Prompt original

> Actualmente la cátedra de base de datos usa la calculadora de algebra relacional relax, podemos copiarle el lenguaje. Acá podés buscar información https://dbis-uibk.github.io/relax/help

Resultado: la página de ayuda no se pudo leer directamente, así que la IA consultó el código fuente de RelaX en GitHub. Identificó los conflictos LL(1) (theta-join y join natural, asignación y consulta, `x` como producto, funciones, tablas con datos) y propuso resolverlos con los resolvers de Coco/R.

Prompt 6 — Alcance según el material de Base de Datos 1

Fecha: 20/10/2026 · **Fase:** Diseño del lenguaje

Prompt original (con las diapositivas de la Unidad 4 de Base de Datos 1 adjuntas)

> group: ClaseAlgebra materia = { codM, nomM, areaConoc 1, 'Base de Datos', 'ISyBD' [...] (el dataset completo, con las tablas materia, alumno, examen y boleta) } Esos son ejemplos que da la materia base de datos 1 en sus diapositivas entregadas por la cátedra, para que veas el alcance que tiene que tener

Resultado: definió el alcance de la versión 1: asignación, ρ, σ, π, ∪, ∩, −, ×, joins (theta, natural y externos), ÷, γ y datasets con datos. Corrigió una recomendación propia anterior (mantener `x` como producto, porque la usa la cátedra) y propuso reescribir la división con operadores primitivos para traducirla a SQL.

Prompt 7 — Gramática de la versión 1 y pruebas

Fecha: 20/10/2026 · **Fase:** Etapa 1: scanner y parser

Prompt original

> Prefiero quedarme con la versión que tiene lo que se utiliza en base de datos. Las demás operaciones son difíciles de implementar en COCO/R usando grámaticas LL1 y no va a ser utilizado en la cátedra de base de datos.

Resultado: la gramática se generó con Coco/R sin avisos LL(1) y pasó las 8 pruebas (2 válidas y 6 inválidas). En las pruebas apareció una limitación: como `x` es palabra reservada, ninguna variable puede llamarse así.

Configuración del entorno y pruebas

Prompt 8 — Descarga de Coco.exe

Fecha: 01/11/2026 · **Fase:** Entorno

Prompt original

> De donde descargo coco.exe?
> joaquin@JOANOTEBOOK:~/Documentos/Compiladores/Algebra Relacional/compilador-ar-etapa1$ mono Coco.exe AR.atg Cannot open assembly 'Coco.exe': No existe el archivo o el directorio.

Resultado: el enlace a la página oficial de Coco/R (SSW, Universidad de Linz) y los comandos para descargar `Coco.exe` y los `.frame`. Advirtió que las plantillas tienen que ser de la misma versión que el ejecutable.

Prompt 9 — Interpretación de la salida de Coco/R

Fecha: 01/11/2026 · **Fase:** Entorno

Prompt original

> mono Coco.exe AR.atg WARNING: The runtime version supported by this application is unavailable. Using default runtime: v4.0.30319 Coco/R (Apr 19, 2011) checking AR deletable parser + scanner generated 0 errors detected

Resultado: confirmó que la generación fue correcta y que la advertencia de mono no afecta el resultado. Indicó correr las pruebas, para lo cual hacía falta instalar .NET.

Prompt 10 — Error al instalar .NET

Fecha: 01/11/2026 · **Fase:** Entorno

Prompt original

> sudo apt install dotnet-sdk-8.0 Error: No se ha podido localizar el paquete dotnet-sdk-8.0 Error: No se pudo encontrar ningún paquete usando «*» con «dotnet-sdk-8.0» ./probar.sh ./probar.sh: línea 3: dotnet: orden no encontrada Falló la compilación

Resultado: dos opciones: buscar la versión de .NET disponible con `apt search`, o compilar con mono (`mcs`) sin instalar .NET.

Prompt 11 — Instalación de .NET 10

Fecha: 01/11/2026 · **Fase:** Entorno

Prompt original

> apt search dotnet-sdk dotnet-sdk-10.0/resolute-updates,resolute-security 10.0.114-0ubuntu1~26.04.1 .NET 10.0 Software Development Kit [...]

Resultado: instalar `dotnet-sdk-10.0` y cambiar `net8.0` por `net10.0` en `AR.csproj`.

Prompt 12 — Resultado de las pruebas de la etapa 1

Fecha: 01/11/2026 · **Fase:** Pruebas

Prompt original

> ./probar.sh [...] (mensaje de bienvenida de .NET) bien pruebas/validas/01_clase_algebra.ra bien pruebas/validas/02_operadores.ra [...] bien pruebas/invalidas/06_cadena_sin_cerrar.ra 0 prueba(s) fallaron

Resultado: confirmó que el entorno estaba completo, explicó el ciclo "regenerar con Coco/R → correr las pruebas", recomendó agregar pruebas con cada cambio y propuso pasar a la etapa del AST.

Etapa del AST y documentación

Prompt 13 — Construcción del árbol de sintaxis abstracta

Fecha: 01/11/2026 · **Fase:** Etapa 2: AST

Prompt original

> La etapa 1 (scanner y parser como reconocedor) funciona y pasa todas las pruebas.
>
> Diseñá las clases del AST y agregá a `AR.atg` los atributos y las acciones semánticas para construirlo. El programa tiene que imprimir el árbol, para verificar la precedencia y la asociatividad de los operadores.

Resultado: el archivo `Ast.cs`, la gramática atribuida y 5 pruebas de árbol, que coincidieron con los árboles esperados. Durante el desarrollo aparecieron tres problemas propios de Coco/R, que se resolvieron:

- Los atributos con tipos genéricos (`List<X>`) tienen que ir entre `<. ... .>`.
- Las clases del AST no pueden llamarse igual que los no terminales.
- Las clases del AST tienen que ser públicas.

Prompt 14 — Archivo de contexto para Claude Code

Fecha: 01/11/2026 · **Fase:** Documentación

Prompt original

> Quiero que generes un archivo .md explicando todo el contexto del proyecto para que se lo de a claude code

Resultado: el archivo `CLAUDE.md`, con el plan detallado para el análisis semántico y la generación de SQL y una lista de las trampas de Coco/R encontradas hasta ese momento.

Etapas 3 y 4: análisis semántico y generación de SQL

Prompt 15 — Unir las etapas, análisis semántico y generador de SQL

Fecha: 01/11/2026 · **Fase:** Etapas 3 y 4: semántico y SQL

Prompt original (enviado a Claude Code)

> Rol: sos un desarrollador experto en compiladores, C# y Coco/R, que sabe explicar lo que hace.
>
> Contexto: es el trabajo integrador de Compiladores, un compilador de algebra relacional (con el lenguje de RelaX reducido a la parte que usa Base de Datos 1) a SQL para SQLite. Lee el CLAUDE.md completo antes de tocar codigo, ahi estan el lenguaje, la gramatica, el AST y detalles de Coco/R. Los archivos estan divididos en dos entregas (etapa 1 y etapa 2), por lo que es posible que haga falta unirlos. Tenes total libertad de cambiar la estructura de carpetas y mover archivos.
>
> Tarea:
>
> 1. Uní las dos etapas en una sola carpeta. Quedate con lo mas nuevo de cada una (ojo con el net10.0 y con las plantillas .frame de Coco.exe), regenera el parser y agrega un .gitignore. El objetivo sería montar la base del repositorio para github.
> 2. Segui con el analisis semantico en Semantico.cs: tabla de simbolos, esquema de salida de cada nodo guardado en el AST y todos los chequeos del CLAUDE.md, con errores en castellano indicando linea y columna.
> 3. Despues hace el generador de SQL en Generador.cs. Cada nodo tiene que ser un SELECT con las columnas de su esquema con el nombre completo ("alumno.cuil"), asi no chocan los nombres despues de un join. La division no existe en SQL, resolvela con EXCEPT. Verificalo ejecutando el SQL en SQLite y comparando con resultados calculados a mano, no con lo que da el compilador.
>
> Cada etapa se debe validar con sus pruebas y ./probar.sh debe correr con 0 fallos. Si algo de lo planeado en el CLAUDE.md no funciona, cambialo y explica por que.
>
> Formato de entrega: un commit por etapa con un mensaje razonable, sin push. Codigo, comentarios, mensajes de error y pruebas en castellano, claro y facil de explicar. Al final de cada etapa contame que hiciste, que decisiones tomaste y que quedo pendiente.

Resultado: unió las etapas 1 y 2 en una sola carpeta (net10.0, plantillas .frame originales y .gitignore). Creó Semantico.cs, con tabla de símbolos, esquemas guardados en el AST y 23 pruebas de errores. Creó Generador.cs, donde cada nodo es un SELECT con nombres

completos, el join natural lleva ON explícito y la división se resuelve con EXCEPT. Verificó 20 consultas en SQLite contra resultados calculados a mano. Quedaron 38 pruebas con 0 fallos y un commit por etapa.